YZT101

while Döngüsü

sayaç • biriktirici • koşula bağlı tekrar • sonsuz döngü • döngü kontrolü

Bu haftanın temel sorusu

“Bir işlemi ne zamana kadar tekrar etmeliyiz?”

5. Hafta • 2 saat teori + 2 saat uygulama

BAŞLANGIÇ

Bu haftanın hedefi

Slaytlar kısa; örnekler ve sınıf anlatımı ayrıntılı olacak.

Tekrarı tanı

Hangi iş tekrar ediyor?

while kur

Koşul ne zamana kadar True?

Değeri güncelle

Döngü nasıl ilerliyor?

Test et

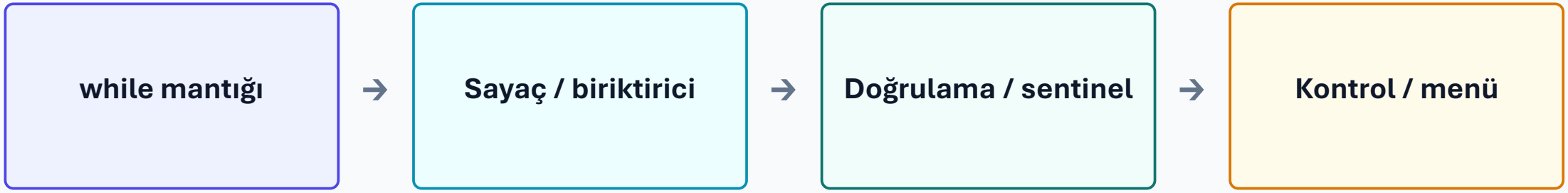
Ne zaman duruyor?

Döngüyü ezberlemeyeceğiz; her turda programın durumunu izleyeceğiz.

BAŞLANGIÇ

4 ders saatlik yol haritası

Her ders saatinde yeni örnekler ve kısa kod okuma etkinlikleri var.



Bu hafta “döngü izleme tablosu” en önemli çalışma yöntemimiz.

HATIRLAMA

Karar ile tekrar arasındaki fark

if

Koşulu kontrol eder
ve bir kez karar verir.

while

Koşulu kontrol eder
ve yeniden dönebilir.

while = koşul True kaldığı sürece tekrar

01

while nasıl çalışır?

Tekrarın çekirdeği: koşul doğru olduğu sürece aynı blok yeniden çalışır.

Hedef: başlangıç, koşul, gövde ve güncelleme ilişkisini bir döngü izleme tablosuyla okuyabilmek.

Neden döngü kullanıyoruz?

Aynı işi tekrar yazmak yerine...

tekrarın kuralını yazarız.

Tekrar eden kod → döngü adayıdır.

while döngüsünün iskeleti

Dört parçayı bul: başlangıç • koşul • gövde • güncelleme

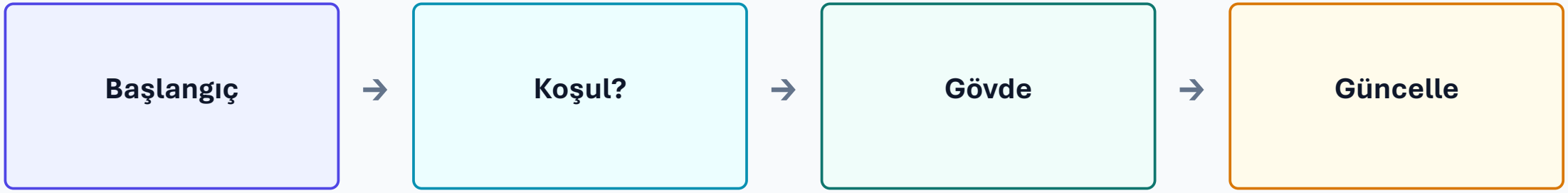
```
sayac = 1

while sayac <= 5:
    print(sayac)
    sayac = sayac + 1
```

Döngünün bitmesi için koşulu etkileyen bir değer değişmelidir.

Döngünün yürütme akışı

False olduğunda döngüden çıkılır; True olduğunda yeni tur başlar.



Gövde bittiğinde program yeniden koşula döner.

Örnek 1: 1’den 5’e kadar

Önce çıktıyı tahmin et, sonra çalıştır.

```
sayac = 1

while sayac <= 5:
    print("Tur:", sayac)
    sayac += 1

print("Bitti")
```

Beklenen: 5 tur + ardından “Bitti”

Döngü izleme tablosu

Kodun her turdaki durumunu görünür hale getirir.

Tur	Başta sayaç	Koşul	Çıktı	Tur sonunda
1	1	1 ≤ 5 → True	1	2
2	2	2 ≤ 5 → True	2	3
3	3	3 ≤ 5 → True	3	4
4	4	4 ≤ 5 → True	4	5
5	5	5 ≤ 5 → True	5	6
—	6	6 ≤ 5 → False	—	Döngü biter

Tablo, “bir kez fazla mı / eksik mi?” hatalarını yakalamada çok etkilidir.

while hiç çalışmayabilir

İlk koşul False ise...

gövdeye bir kez bile girilmez.

while önce koşulu kontrol eder.

Örnek 2: Geri sayım

Döngüler yalnızca artmak zorunda değildir.

```
sayac = 5

while sayac >= 1:
    print(sayac)
    sayac -= 1

print("Başla!")
```

Güncelleme bu kez -1: koşulu False’a doğru ilerletiyoruz.

Güncelleme miktarı sonucu değiştirir

+1

1, 2, 3, 4, 5

+2

1, 3, 5

Başlangıç + adım + koşul birlikte sonucu belirler.

Tahmin et → çalıştır → karşılaştır

Kodu çalıştırmadan önce üç soruya cevap ver.

```
x = 2

while x < 10:
    print(x)
    x += 3
```

Kaç tur? • Hangi değerler? • Döngü sonunda x kaç?

Hata avı: Güncelleme unutuldu

Kod sözdizimsel olarak doğru olabilir, ama mantıksal olarak bitmeyebilir.

```
sayac = 1

while sayac <= 5:
    print(sayac)
    # sayac değişmiyor!
```

Koşul hep 1 ≤ 5 → True

02

Sayaç ve biriktirici

Tekrarın içinde iki temel rol: “kaç tane?” ve “toplam ne kadar?”

Hedef: sayaç ile biriktiriciyi ayırmak ve aynı döngüde birlikte kullanabilmek.

Sayaç neyi tutar?

“Kaç kez oldu?”

Genellikle 0’dan başlar ve belirli olayda artar.

Sayaç değer toplamaz; olay sayar.

Örnek 3: Deneme sayacı

Kullanıcı doğru PIN'i bulana kadar deneme sayısını tutalım.

```
pin = "2468"
giris = ""
deneme = 0

while giris != pin:
    giris = input("PIN: ")
    deneme += 1

print("Deneme sayısı:", deneme)
```

Sayaç, döngünün kaç kez çalıştığını gözlememizi sağlar.

Biriktirici neyi tutar?

“Toplam ne kadar?”

Her turdaki değeri mevcut toplamın üzerine ekler.

```
toplam = toplam + yeni_deger
```

Örnek 4: 1’den 5’e toplam

Sayaç ilerlerken toplam da biriksin.

```
sayac = 1
toplam = 0

while sayac <= 5:
    toplam += sayac
    sayac += 1

print("Toplam:", toplam)
```

1 + 2 + 3 + 4 + 5 = 15

İzleme: toplam nasıl büyür?

Her turda eski toplam korunur ve yeni değer eklenir.

Tur	sayaç	önce toplam	eklenen	sonra toplam
1	1	0	+1	1
2	2	1	+2	3
3	3	3	+3	6
4	4	6	+4	10
5	5	10	+5	15

Biriktirici, önceki turların bilgisini taşır.

Örnek 5: Kullanıcıdan 3 sayı al

Tekrar + input + toplam

```
adet = 0
toplam = 0

while adet < 3:
    sayi = int(input("Sayı: "))
    toplam += sayi
    adet += 1

print("Toplam:", toplam)
```

Döngü 3 kez veri alır; toplam girilen değerlere bağlıdır.

Ortalama için iki bilgi gerekir

Toplam + Adet

Ortalama = toplam / adet

Sayaç ve biriktirici aynı programda birlikte çalışabilir.

Örnek 6: Yalnızca başarılıları say

Döngü her tur çalışır; sayaç yalnızca koşulda artar.

```
adet = 0
basarili = 0

while adet < 4:
    puan = int(input("Puan: "))
    if puan >= 50:
        basarili += 1
    adet += 1

print("Başarılı:", basarili)
```

while + if birlikte: tekrarın içinde karar.

Sınıf uygulaması

5 harcama al → toplamı ve 100 TL üstü kaç tane olduğunu yazdır.

İki değişken rolü: biriktirici + koşullu sayaç

Önce algoritmayı söyle, sonra kodu yaz.

Çözüm: Harcama özeti

Aynı döngüde biriktirici ve koşullu sayaç.

```
adet = 0
toplam = 0
yuksek = 0

while adet < 5:
    harcama = float(input("Harcama: "))
    toplam += harcama
    if harcama > 100:
        yuksek += 1
    adet += 1

print(f"Toplam: {toplam:.2f} TL")
print("100 TL üstü:", yuksek)
```

Bir döngü, birden fazla amacı aynı anda gerçekleştirebilir.

03

Koşula bağlı tekrar

Kaç tur olacağını baştan bilmiyorsak bitiş kuralını tanımlarız.

Hedef: veri doğrulama, sentinel ve kullanıcı kontrollü tekrar örneklerini kurabilmek.

Amaç: geçerli veri elde etmek

Geçersiz olduğu sürece yeniden sor.

while koşulu “hata durumu”nu temsil edebilir.

İlk giriş geçerliyse döngü hiç çalışmayabilir.

Örnek 7: Yaş doğrulama

0–120 aralığı dışında kaldığı sürece yeniden sor.

```
yas = int(input("Yaş: "))

while yas < 0 or yas > 120:
    print("Geçersiz yaş.")
    yas = int(input("Yaş: "))

print("Kabul edilen yaş:", yas)
```

Koşul “geçersizliği” tanımlar.

Örnek 8: Puan doğrulama

0 ile 100 dahil.

```
puan = int(input("Puan: "))

while not (0 <= puan <= 100):
    print("Puan 0-100 arasında olmalı.")
    puan = int(input("Puan: "))

print("Puan kaydedildi.")
```

Aynı kural farklı Boolean ifadelerle yazılabilir.

Doğrulama döngüsünde iki kritik nokta

Koşul

Geçersiz durumu doğru tanımla.

Güncelleme

Döngü içinde yeni veri al.

Koşul doğru ama veri değişmiyorsa döngü bitmez.

Özel bir değer döngüyü bitirebilir

Örn: 0 = bitir

Bu değere “sentinel / nöbetçi değer” diyebiliriz.

Sentinel, normal verilerden ayırt edilebilir olmalıdır.

Örnek 9: 0 girilene kadar toplam

Kaç sayı geleceği baştan bilinmiyor.

```
toplam = 0
sayi = int(input("Sayı (bitir: 0): "))

while sayi != 0:
    toplam += sayi
    sayi = int(input("Sayı (bitir: 0): "))

print("Toplam:", toplam)
```

0, döngü gövdesine girmez; yalnızca bitiş sinyalidir.

Örnek 10: Sentinel + sayaç + ortalama

Bitiş sayısı bilinmiyor; program kendisi sayıyor.

```
toplam = 0
adet = 0
sayi = int(input("Sayı (bitir: 0): "))

while sayi != 0:
    toplam += sayi
    adet += 1
    sayi = int(input("Sayı (bitir: 0): "))

if adet > 0:
    print("Ortalama:", toplam / adet)
```

while tekrar toplar; if sıfıra bölmeyi önler.

Her veri işlenmek zorunda mı?

Bazen bir turu atlamak isteriz.

`continue` → doğrudan sonraki koşul kontrolüne geçer.

continue kullanırken güncellemenin atlanmamasına dikkat.

Örnek 11: Negatifleri atla

continue ile seçili turları işlem dışı bırakma.

```
sayac = 0
toplam = 0

while sayac < 5:
    sayi = int(input("Sayı: "))
    sayac += 1

    if sayi < 0:
        continue

    toplam += sayi

print("Pozitiflerin toplamı:", toplam)
```

Sayaç continue'den önce güncelleniyor.

04

Sonsuz döngü ve kontrol

Döngü bitmiyorsa ya hata vardır ya da bitişi içeriden yönetiyoruz.

Hedef: sonsuz döngü nedenlerini teşhis etmek, break kullanmak ve menü tabanlı küçük program kurmak.

Bir döngü neden bitmez?

Koşul hiçbir zaman False olmuyorsa...

döngü devam eder.

Sor: “Koşulu False yapacak şey nerede?”

Üç yaygın neden

Kod kısa olabilir; hata yine de mantıksaldır.

1 Değişken değişmiyor

2 Yanlış yönde değişiyor

3 Güncelleme bazı yollarda atlanıyor

Her durumda aynı teşhis: koşul + güncelleme ilişkisini izle.

while True ne demek?

Koşul bilinçli olarak her zaman True.

```
while True:
    komut = input("Komut (çıkış: q): ")

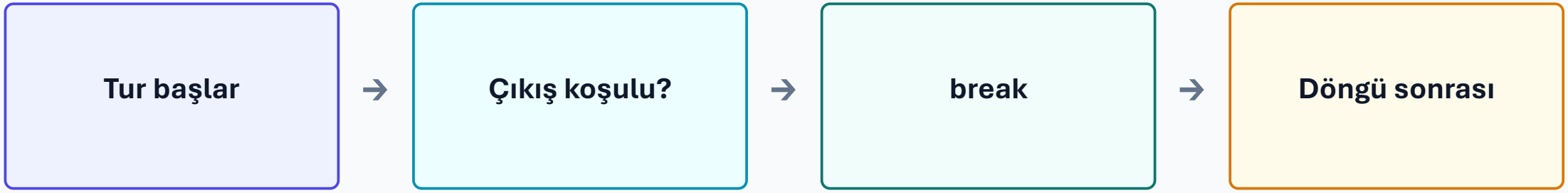
    if komut == "q":
        break

    print("Komut işlendi:", komut)
```

Bitiş koşulu dışarıda değil, gövdenin içinde `break` ile yönetiliyor.

break çalıştığında ne olur?

break, o turdaki kalan satırları da atlar.



break yalnızca içinde bulunduğu döngüyü sonlandırır.

Menü neden while ister?

Kullanıcı çıkana kadar tekrar göster.

Her tur: göster → seçim al → işle → tekrar

Menü, koşula bağlı tekrar için gerçekçi bir örnektir.

Örnek 12: Basit işlem menüsü

Kullanıcı 0 seçeneğine kadar menü açık.

```
while True:
    print("1 - Merhaba de")
    print("2 - Saati göster (örnek)")
    print("0 - Çıkış")

    secim = input("Seçim: ")

    if secim == "0":
        break
    elif secim == "1":
        print("Merhaba!")
    elif secim == "2":
        print("Saat özelliği daha sonra eklenecek.")
    else:
        print("Geçersiz seçim.")
```

while tekrar eder, if/elif seçimin davranışını belirler.

Örnek 13: Bakiye menüsü

Durum döngüler arasında korunabilir.

```
bakiye = 0.0

while True:
    print("1-Ekle  2-Göster  0-Çıkış")
    secim = input("Seçim: ")

    if secim == "0":
        break
    elif secim == "1":
        tutar = float(input("Tutar: "))
        bakiye += tutar
    elif secim == "2":
        print("Bakiye:", bakiye)
    else:
        print("Geçersiz seçim")
```

bakiye, turlar arasında biriktirilen program durumudur.

Hata: Biriktirici yanlış yerde

Bakiye neden hiç büyümüyor?

```
while True:
    bakiye = 0.0   # Her tur sıfırlanıyor!
    secim = input("1-Ekle, 0-Çıkış: ")

    if secim == "0":
        break

    tutar = float(input("Tutar: "))
    bakiye += tutar
    print("Bakiye:", bakiye)
```

Başlangıç satırının konumu, değişkenin ömrünü etkiler.

Koşullu while mı, while True + break mi?

Koşullu while

Devam kuralı baştan netse
niyet açık görünür.

while True + break

Çıkış kararı gövdenin
içinde doğalysa pratik olabilir.

Amaç “en kısa kod” değil, en anlaşılır akış.

while için 5 soruluk kontrol

Döngü yanlışsa önce bu beş noktayı kontrol et.

1. Başlangıç değeri?

2. Koşul ne zaman True?

3. Ne değişiyor?

4. Değişim doğru yönde mi?

5. False olabilecek mi?

Off-by-one: bir kez fazla / eksik

sayac < 5

1, 2, 3, 4

sayac <= 5

1, 2, 3, 4, 5

Sınır değer operatörü tur sayısını değiştirir.

Güncellemenin yeri çıktıyı değiştirir

Önce yazdır

```
print(sayac)
sayac += 1
```

Önce artır

```
sayac += 1
print(sayac)
```

Aynı koşul, farklı sıra → farklı çıktı.

Mini proje: Kasa uygulaması

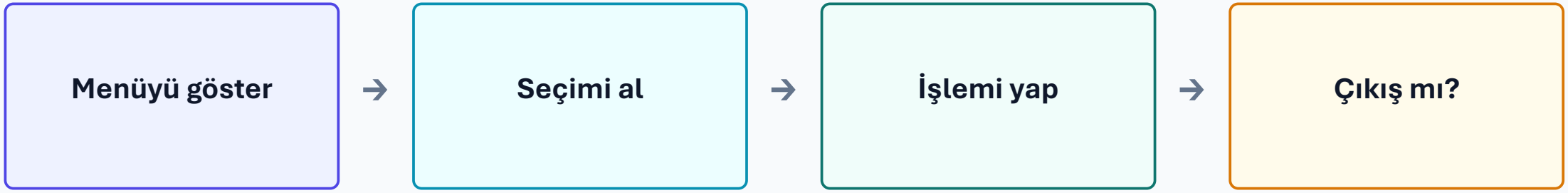
Para ekle • harcama yap • bakiye göster • çık

while + if/elif + doğrulama + biriktirici

Önce akışı kuracağız; sonra kodu parça parça yazacağız.

Kasa uygulamasının akışı

Çıkış değilse yeni tur başlar.



bakiye değişkeni döngünün dışında başlatılır.

Kasa uygulaması — iskelet

Önce menü ve çıkış.

```
bakiye = 0.0

while True:
    print("\n1-Para ekle")
    print("2-Harcama yap")
    print("3-Bakiye göster")
    print("0-Çıkış")

    secim = input("Seçim: ")

    if secim == "0":
        print("Program kapatılıyor.")
        break
```

Önce çalışan iskelet, sonra seçenekleri ekle.

Kasa uygulaması — para ekle

Biriktirici mantığı menünün içinde.

```
elif secim == "1":
    tutar = float(input("Eklenecek tutar: "))

    if tutar > 0:
        bakiye += tutar
        print("Yeni bakiye:", bakiye)
    else:
        print("Tutar pozitif olmalı.")
```

bakiye += tutar → değer yeni turlara taşınır.

Kasa uygulaması — harcama

İki koşul: pozitif ve bakiye yeterli.

```
elif secim == "2":
    tutar = float(input("Harcama tutarı: "))

    if tutar <= 0:
        print("Tutar pozitif olmalı.")
    elif tutar > bakiye:
        print("Yetersiz bakiye.")
    else:
        bakiye -= tutar
        print("Yeni bakiye:", bakiye)
```

Döngü tekrar sağlar; karar yapısı güvenli işlem seçer.

Test senaryosu

+100 → -30 → bakiye göster → çık

Beklenen bakiye: 70

Tek bir mutlu yol yeterli değil; geçersiz girişleri de test et.

UYGULAMA

Bireysel görev

Sayı tahmin oyunu

Gizli sayı bulunana kadar tahmin al • deneme sayısını yazdır

while + if/elif + sayaç

KAPANIŞ

Bugünün altı ana fikri

Slaytta kısa; açıklamasını siz yapabiliyor olmalısınız.

Koşul

Güncelleme

Sayaç

Biriktirici

Sentinel

break

Bir while döngüsünü açıklayabiliyorsanız yazmaya da hazırsınız.

KAPANIŞ

Ders çıkış bileti

Kod yazmadan, üç cümleyle cevapla.

1. while ile if arasındaki temel fark nedir?

2. Sonsuz döngüyü önlemek için hangi üç şeyi kontrol edersin?

3. Sayaç ile biriktirici arasındaki fark nedir?

Cevabın koddan bağımsız olarak kavramı açıklayabilmeli.

Hafta sonu mini pratik

ATM PIN denemesi

Doğru PIN’e kadar sor • deneme say • 3 yanlışta kilitle

Zorunlu çözüm kalıbı yok; önce algoritma kur.

Sonraki hafta

for döngüsü ve range()

Belirli sayıda tekrar • koleksiyon üzerinde yineleme • iç içe döngüler • for / while seçimi

Bugünün sorusu: “Ne zamana kadar?”
Gelecek haftanın sorusu: “Kaç kez / hangi elemanlar üzerinde?”